ONLINE FOOD DELIVERY SYSTEM

Chandan Mondal (202303103510104)
AMTICS, UTU

1. Problem Statement

Manual or phone-based food ordering can cause wrong orders, delayed deliveries, and poor order tracking. The proposed Food Delivery System allows customers to browse restaurants, place orders, pay online, and track delivery. Restaurants manage menus and orders, and delivery agents handle dispatch through a centralized system.

Goal: Apply Object-Oriented Design (OOD) to create a modular, reliable, and maintainable food ordering application.

2. System Objectives

- Search restaurants by cuisine, rating, and location.
- View menus and prices before ordering.
- Place, update, and cancel orders.
- Maintain customer and order information.
- Process payments and confirm orders.
- Allow restaurants to manage menu items.

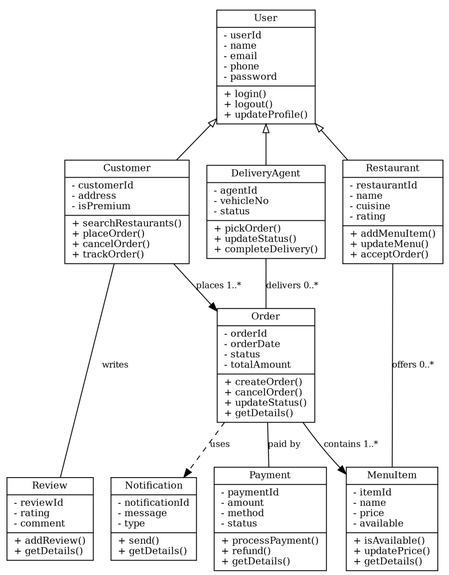
3. Core OOD Principles

Encapsulation – Keep data and related operations within classes.
Abstraction – Hide internal complexity behind simple operations.
Inheritance – Customer, Restaurant, and DeliveryAgent share common properties through a User class.
Polymorphism – Different payment methods (UPI, card, cash) can implement the same payment operation.

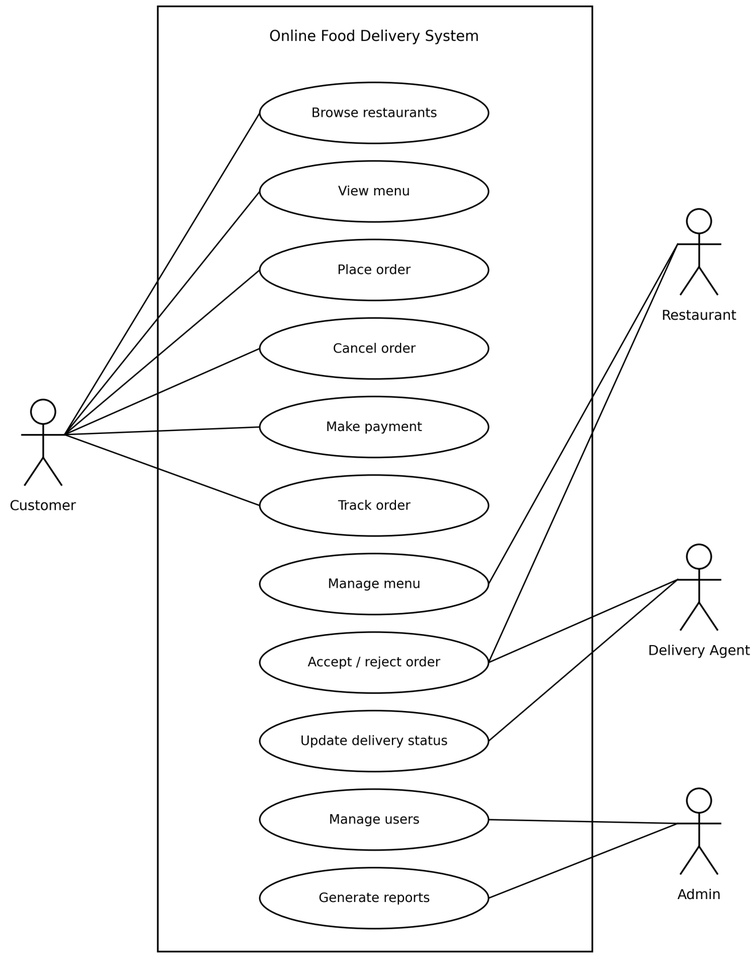
4. Main Classes

User – Common user ID and contact details.
Customer – Searches food and manages orders.
Restaurant – Updates menu items and availability.
DeliveryAgent – Picks up orders and updates delivery status.
Order – Order ID, items, status, and total amount.
Payment – Payment amount, method, and status.

5. UML Class Diagram



6. Use Case Diagram



7. System Architecture

Presentation Layer: Provides the interface for customers, restaurants, delivery agents, and administrators.
Business Logic Layer: Handles menu search, order processing, payments, and delivery assignment.
Data Layer: Stores customer, restaurant, menu, order, and payment information.
Purpose: Separating these layers improves organization, testing, and maintainability.

8. Class Relationships

- **Inheritance:** Customer, Restaurant, and DeliveryAgent inherit shared properties from User.
- **Association:** A Customer places orders.
- **One-to-many:** A Customer may place multiple orders over time.
- **Order–MenuItem:** An order contains one or more menu items.
- **Order–Payment:** An order is associated with one payment record.

9. Order Workflow

1. Customer browses restaurants and selects food items.
2. System checks item availability.
3. Customer reviews the cart and places the order.
4. Restaurant accepts and prepares the order.
5. Customer completes the payment.
6. Delivery agent picks up and delivers the order.
7. Order status is updated to delivered.

10. Quality Assurance

- **Functional testing:** Test searching, ordering, payment, and cancellation.
- **Availability testing:** Prevent ordering items that are out of stock.
- **Validation:** Check customer details and delivery address.
- **Security:** Protect personal data and restrict access by role.
- **Integration testing:** Verify order, payment, and delivery modules work together.

11. Impact and Future Scope

Benefits

- Faster ordering and restaurant management.
- Fewer manual errors and wrong orders.
- Easier maintenance, testing, and extension.

Future Enhancements

- AI-based food recommendations.
- Cloud-based data management.
- Mobile applications and live location tracking.
- Secure online payment integration.

12. Limitations and Challenges

- Handling heavy order traffic during peak hours.
- Protecting customer information and payment details.
- Maintaining accurate menu availability and delivery times.

Conclusion

Object-Oriented Design separates food delivery operations into reusable classes and clear relationships, supporting a reliable, extensible, and maintainable ordering system.